

CloudShield AI / DAIL

04 — Terraform & AWS Technical Specification

Version 1.0 | Controlled Infrastructure Semantics Baseline | 1 September 2026

Purpose: Define the exact Terraform/HCL and AWS infrastructure semantics that the DAIL MVP is permitted to parse, normalize, compare, graph, verify, and use in candidate promotion. This document deliberately constrains the supported infrastructure model; unsupported semantics must never be silently treated as safe or proven.

Document Control

Field	Value
Document ID	CSAI-DAIL-TFAWS-004
Status	FINAL TECHNICAL BASELINE
Parent documents	01 SRS; 02 System Architecture; 03 DAIL Functional Specification
IaC format	Terraform/HCL controlled subset
Cloud provider	AWS
Primary tooling	Terraform CLI + controlled HCL tooling
Primary internal representation	Deterministic normalized resource model
Initial resource scope	VPC, subnet, route table/association, internet gateway, EC2, security groups/rules, RDS
Design principle	Explicit supported semantics; conservative unknown/unsupported handling

1. Purpose and Technical Scope

This specification establishes the infrastructure vocabulary DAIL understands in the MVP. It defines supported resources, relevant attributes, normalization rules, references, dependency semantics, security semantics, functional semantics, plan/state interpretation boundaries, unsupported cases, fixture conventions, and AWS runtime requirements.

The specification is not a general Terraform manual and does not attempt to model the complete AWS control plane. It defines a deliberately small semantic subset sufficient to implement and test the DAIL research mechanism.

2. Supported Resource Set

Resource	Primary purpose in MVP	Priority
aws_vpc	Network boundary and address space.	MUST
aws_subnet	Placement/network segmentation for compute/database.	MUST
aws_route_table	Routing behavior.	MUST
aws_route_table_association	Associates subnet with route table.	MUST
aws_internet_gateway	Internet connectivity for VPC routing.	MUST
aws_instance	Application/compute endpoint.	MUST
aws_security_group	Stateful network access policy.	MUST
aws_security_group_rule	Explicit ingress/egress rule	MUST

	representation.	
aws_db_instance	RDS database endpoint for functional invariant scenarios.	MUST

3. Supported Terraform Configuration Model

The MVP should prefer explicit resource blocks and simple references. Complex meta-programming is not part of the semantic contract.

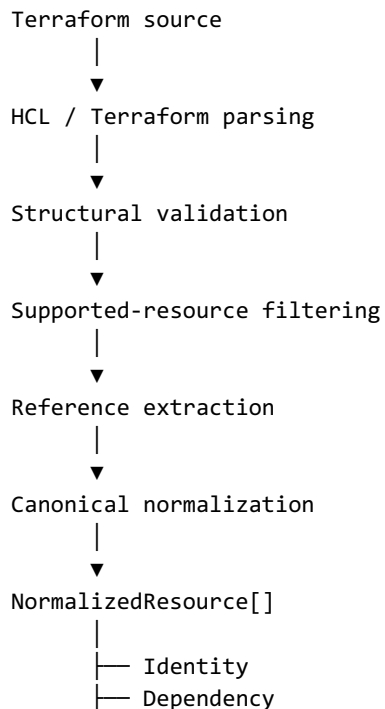
3.1 Preferred Constructs

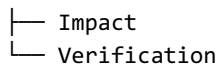
- resource blocks for the supported resource types.
- Explicit attribute assignments.
- Direct resource references.
- Simple lists/maps/sets where their semantics are known.
- Variables/locals only when their resolved values can be represented deterministically.
- Explicit security-group rules.
- Controlled module use only if expanded into a supported normalized representation before DAIL analysis.

3.2 Restricted/Unsupported Constructs

- Unknown provider semantics outside AWS.
- Dynamic blocks whose resulting supported semantics cannot be deterministically resolved.
- Complex expressions that cannot be resolved safely.
- Arbitrary provisioners and external side effects.
- Unsupported resources.
- Unresolved data-source semantics when required information is unavailable.
- Terraform features whose semantics cannot be mapped to the controlled normalized model.

4. Terraform Processing Pipeline





Terraform CLI may be used as an authoritative structural tool where appropriate. DAIL's normalized semantic model remains the internal representation used by the research core.

5. Normalized Resource Contract

```
NormalizedResource {
  address: string
  resource_type: string
  logical_identity: object
  attributes: object
  references: list[Reference]
  security_attributes: object
  semantic_attributes: object
  provenance: object
  canonical_json: string
  fingerprint: string
}
```

5.1 Address

The Terraform resource address identifies the source-level resource within the supported configuration. Addresses must be normalized consistently before comparison.

5.2 Resource Type

The resource type is the exact supported Terraform resource type and is part of the semantic identity context.

5.3 Attributes

Only relevant attributes need to be included in the canonical semantic representation. Irrelevant formatting and ordering must not affect the fingerprint.

5.4 Provenance

- Source file/path where available.
- Terraform resource address.
- Module path where supported.
- Parser/tool version.
- Normalization version.

6. Canonicalization Rules

6.1 General Rules

- Canonical output must be deterministic.

- Map/object keys are sorted.
- Set-like values are represented deterministically.
- Equivalent supported rule forms use the same semantic representation.
- Null/absent/default handling follows one documented normalization policy.
- Strings are normalized only where semantics permit; arbitrary user data must not be altered.
- Resource references use canonical addresses.
- Fingerprint input is canonical semantic data, not raw HCL text.

6.2 Fingerprinting

```
canonical = deterministic_serialize(normalized_semantics)
fingerprint = SHA-256(canonical)
```

The exact serialization implementation may be Python JSON with deterministic ordering, provided it is stable across runs and documented. Fingerprint changes must correspond to relevant semantic changes within the supported model.

7. Resource-Specific Semantics

7.1 aws_vpc

Field	Semantic role
cidr_block	Primary VPC address space.
enable_dns_support	DNS behavior where used by supported functional semantics.
enable_dns_hostnames	DNS hostname behavior where used.
tags	Metadata; not security/functional identity unless explicitly used by policy.
id/reference	Used for dependency relationships.

MVP verification should not infer broader AWS VPC semantics from fields not explicitly represented.

7.2 aws_subnet

Field	Semantic role
vpc_id	Parent VPC relationship.
cidr_block	Subnet address range.
availability_zone	Placement information where relevant.
map_public_ip_on_launch	Important public-addressing behavior for security analysis.
route_table association	Derived through aws_route_table_association relationship.
tags	Metadata unless explicitly used.

7.3 aws_route_table

Field	Semantic role
vpc_id	Parent VPC relationship.
route	Routing destination/target semantics within supported subset.
gateway_id	Internet gateway route target when supported.
nat_gateway_id	NAT target only if explicitly included in the controlled model.
tags	Metadata.

7.4 aws_route_table_association

Field	Semantic role
subnet_id	Connects subnet to route table.
route_table_id	Selected routing table.

The association creates a dependency edge used to infer effective subnet routing.

7.5 aws_internet_gateway

Field	Semantic role
vpc_id	Associates gateway with VPC.
attachment state	Runtime/provider state where available; not assumed unless observed/verified.

7.6 aws_instance

Field	Semantic role
subnet_id	Placement and subnet relationship.
security_group_ids	Network policy relationship.
vpc_security_group_ids	Network policy relationship where used.
associate_public_ip_address	Public addressing semantics.
private_ip	Endpoint addressing when relevant.
public_ip	Runtime-derived endpoint; should not be treated as stable configuration identity.
instance_type	Compute property; usually not directly relevant to network invariants.
tags	Metadata.

7.7 aws_security_group

Field	Semantic role
vpc_id	Security group scope.
name/name_prefix	Human/provider naming; not sufficient identity alone.
description	Metadata.
ingress/egress	Security policy only when inline forms are supported/normalized.
tags	Metadata.

7.8 aws_security_group_rule

Field	Semantic role
security_group_id	Owning security group.
type	ingress/egress direction.
from_port / to_port	Port range.
protocol	Network protocol.
cidr_blocks	IPv4 sources/destinations.
ipv6_cidr_blocks	IPv6 sources/destinations.
security_group_id(s)	SG-to-SG source/destination relationship when supported.
description	Metadata/evidence aid.

7.9 aws_db_instance

Field	Semantic role
engine	Database engine where needed.
port	Functional endpoint port.
vpc_security_group_ids	Network access policy.
db_subnet_group_name	Subnet placement context where supported.

address	Runtime endpoint; used only where functional verification requires it.
identifier	Provider-level name; not alone a stable identity guarantee.
availability_zone	Placement where relevant.

8. Security Group Rule Canonical Model

Security-group rules are central to the MVP security scenarios. Inline rules and separate `aws_security_group_rule` resources must be converted into one semantic rule model when both forms are supported.

```
CanonicalSecurityRule {
  direction: INGRESS | EGRESS
  protocol: tcp | udp | icmp | -1 | ...
  from_port: integer | null
  to_port: integer | null
  ipv4_sources: sorted list
  ipv6_sources: sorted list
  source_security_groups: sorted list
  target_security_group: canonical id
  description: optional metadata
}
```

8.1 Canonical Equivalence

Ordering of CIDRs or equivalent set-like values must not create a semantic difference. A rule allowing 0.0.0.0/0 on TCP/22 must remain semantically recognizable regardless of list ordering.

8.2 Public SSH Predicate

```
Public SSH exposure exists when:
direction = INGRESS
AND protocol permits TCP
AND port range includes 22
AND source includes 0.0.0.0/0
```

IPv6 public exposure may be added as an explicit predicate if included in the locked invariant scope; it must not be silently inferred as covered by the IPv4 predicate.

9. Functional Connectivity Semantics

The canonical functional scenario protects connectivity from an EC2 application instance to an RDS database on TCP/5432.

```
EC2 → RDS TCP/5432 is considered reachable only when
the selected verification method establishes:
1. source EC2 placement/context,
2. destination RDS endpoint/context,
3. relevant security-group policy permits the flow,
4. relevant network routing/placement prerequisites hold,
5. the chosen runtime or formal verifier establishes the required reachability.
```

Configuration inspection alone may establish some prerequisites but must not be represented as runtime reachability unless the verification contract explicitly defines that equivalence.

10. Dependency Semantics

Relationship	Graph meaning
EC2 →Subnet	Instance is placed in subnet.
Subnet →VPC	Subnet belongs to VPC.
Route Table Association →Subnet	Association controls subnet routing context.
Route Table →VPC	Route table belongs to VPC.
Route →Internet Gateway	Route may provide internet path when target/destination satisfy supported rules.
EC2 →Security Group	Instance is governed by SG.
RDS →Security Group	Database is governed by SG.
Security Group Rule →Security Group	Rule belongs to SG.
EC2 →RDS Functional Invariant	Functional invariant scope relates source/destination resources.
Security Group Rule →Security Invariant	Security predicate scope relates rule to protected obligation.

The graph must retain enough provenance to explain why an edge exists.

11. Identity-Relevant Infrastructure Fields

Terraform address is useful source identity but is not sufficient in every replacement scenario. Identity logic is defined more deeply in Document 07; this specification defines which infrastructure facts are available to that engine.

Resource	Potential identity/change evidence
VPC	Address + type + controlled provider identity.
Subnet	Address + type + VPC relationship + controlled identifiers.
Route table	Address + type + VPC relationship + controlled identifiers.
EC2	Address + type + relevant provider ID/runtime identity where available.
Security group	Address + type + VPC scope + provider identity where available.
Security group rule	Owning SG + canonical rule semantics + controlled rule identity.
RDS	Address + type + controlled provider identity + relevant endpoint context.

12. Terraform Plan/State Boundary

12.1 Plan

Terraform plan is useful for identifying intended creates/updates/deletes/replacements within the supported model. DAIL shall not assume that a plan alone proves runtime functional behavior.

12.2 State

Terraform state may provide provider-assigned identifiers and resolved attributes useful for identity and verification. State contents are external inputs and must be treated according to the evidence/versioning policy.

12.3 Configuration

Raw configuration is the primary source for candidate construction. Plan/state augment it when their use is explicitly required by a verifier or identity rule.

13. AWS Runtime Semantics

When live AWS verification is required, DAIL operates against a controlled AWS environment matching the Terraform fixture. AWS is accessed through boto3 or another explicit adapter; domain logic does not call boto3 directly.

13.1 Required AWS Concepts

- VPC and subnet relationships.
- Route table associations.
- Internet gateway attachment/routing where used.
- EC2 network placement and security groups.
- RDS placement/security groups/endpoint.
- Security-group ingress/egress behavior.
- Runtime connectivity for selected functional checks.

13.2 Runtime Evidence

- AWS account/environment identifier in redacted or safe form.
- Resource identifiers needed for traceability.
- Relevant API responses/derived facts.
- Verifier timestamp.
- Verifier version/configuration.
- Failure/error details when access is unavailable.

14. AWS Credential and Environment Rules

- Use least-privilege credentials.
- Credentials must be supplied through environment/secret mechanisms, never source-controlled.
- Use a dedicated controlled account/project/environment for experiments.
- Do not deploy arbitrary LLM-generated infrastructure into production.
- Apply cleanup policy for ephemeral test resources.
- Record enough environment metadata to distinguish runs without exposing secrets.

15. Fixture Architecture

```
terraform/  
└─ fixtures/  
    ├── canonical_regression/  
    │   ├── baseline/  
    │   ├── patch_safe/  
    │   └─ patch_regression/  
    ├── identity/  
    ├── dependency/  
    ├── impact/  
    ├── verification/  
    └─ adversarial/
```


15.1 Canonical Baseline

The canonical baseline should contain the initial security weakness and a working EC2→RDS TCP/5432 functional invariant.

15.2 Regression Patch

The canonical regression patch fixes the security weakness while breaking the protected functional path.

15.3 Safe Patch

The safe patch fixes the security weakness while preserving the functional path.

16. Normalization Examples

16.1 Security Rule

Raw semantics:

```
ingress tcp 22 from ["0.0.0.0/0"]
```

Canonical:

```
{
  "direction": "INGRESS",
  "protocol": "tcp",
  "from_port": 22,
  "to_port": 22,
  "ipv4_sources": ["0.0.0.0/0"],
  "ipv6_sources": [],
  "source_security_groups": []
}
```

16.2 Resource Reference

```
aws_instance.app.subnet_id = aws_subnet.app.id
```

Canonical reference:

```
source = aws_instance.app
attribute = subnet_id
target = aws_subnet.app
```

17. Unsupported and Unknown Semantics

Condition	Required classification
Resource type not in supported set	UNSUPPORTED
Required attribute cannot be resolved	UNKNOWN or UNSUPPORTED depending on cause.
Complex expression cannot be safely interpreted	UNKNOWN/UNSUPPORTED; never optimistic.
AWS API unavailable during required runtime verification	VERIFIER_ERROR or UNKNOWN according to verifier contract.
Provider behavior not represented by model	UNSUPPORTED.
Conflicting identity evidence	UNCERTAIN.
Missing dependency knowledge	UNCERTAIN/UNKNOWN relationship, not proof of no edge.

18. Technical Validation Rules

- Terraform syntax must be valid before normalization succeeds.
- Supported resource type must be recognized.
- Required semantic fields must have valid types.
- Canonical rule ports must obey supported port semantics.
- CIDRs must be syntactically valid before being used as security evidence.
- Resource references must resolve within the supported model when required.
- Duplicate semantically equivalent security rules must not create inconsistent semantics.
- Canonical fingerprints must be stable across repeated normalization.

19. Terraform Tooling Contract

Operation	Expected behavior
validate	Check configuration syntax/structural validity.
plan	Compute intended infrastructure transition where environment permits.
show	Expose machine-readable plan/state information where required.
state	Provide controlled provider identifiers/state information where required.
fmt	Optional development formatting; not a semantic verifier.

The exact Terraform CLI version must be pinned/recorded for reproducible experiments. Tool output must be captured when it is used as evidence.

20. Security Verification Inputs

The security verifier may consume the normalized model and, where required, resolved AWS state. It must have explicit input contracts rather than scraping arbitrary Terraform text.

Security fact	Possible source
Ingress rule	Normalized security-group rule / AWS runtime.
Public source	Canonical CIDR/SG source representation.
Instance SG membership	Normalized resource / AWS runtime.
RDS SG membership	Normalized resource / AWS runtime.
Public addressing	Subnet/instance configuration and/or AWS runtime.
Internet route	Route table + gateway relationship and/or AWS runtime.

21. Functional Verification Inputs

Functional verification for EC2–RDS TCP/5432 may combine configuration-derived topology with runtime evidence. The exact verifier implementation is specified later; this document defines the infrastructure facts that must be available.

- EC2 identity and network placement.
- RDS identity and endpoint/context.
- Source/destination security groups.
- Relevant ingress/egress rules.
- Subnet/VPC relationship.
- Route table association.

- Relevant routes/gateway relationships.
- Runtime connectivity evidence when required.

22. Infrastructure Versioning

The normalized infrastructure model must be versioned alongside normalization rules. A change to canonicalization can change fingerprints and therefore must be visible in evidence.

Versioned item	Requirement
Terraform CLI	Record exact version.
Normalization schema	Record version.
Canonicalization algorithm	Record version.
Supported resource semantics	Record specification/model version.
AWS verifier	Record implementation/version.
Fixture	Record fixture identifier/version/commit.

23. Technical Test Matrix

Area	Representative tests
Parsing	Valid/invalid HCL; supported/unsupported resources.
Canonicalization	Ordering invariance; equivalent SG forms; stable serialization.
Fingerprint	Same semantics → same fingerprint; relevant change → different fingerprint.
References	Correct target resolution and dependency extraction.
SG rules	SSH public rule detection; non-SSH/non-public cases.
Topology	EC2/subnet/VPC/route associations.
Identity inputs	Provider ID/address combinations and replacement evidence.
AWS adapter	Mocked API responses, access failure, missing resource.
Fixtures	Canonical regression and safe promotion scenarios.

24. Technical Acceptance Criteria

- All nine MVP resource types can be parsed and normalized within the declared subset.
- Equivalent supported security-rule representations canonicalize consistently.
- Canonical fingerprints are deterministic.
- Supported Terraform references become explicit dependency facts.
- Unsupported resources never silently become normalized as known-safe semantics.
- The canonical public-SSH predicate is machine-detectable.
- The canonical EC2–RDS TCP/5432 topology can be represented.
- Terraform CLI version and normalization version are recorded.
- A controlled AWS adapter can provide runtime evidence when required.
- Canonical regression and safe fixtures can be constructed from the defined resource subset.

25. Implementation Boundary

This specification deliberately does not define the complete algorithms for identity, impact, verification, or promotion. It defines the infrastructure facts and semantics those later components are allowed to consume. The next specifications must preserve this boundary.

25.1 What this document owns

- Terraform/HCL interpretation within the controlled subset.
- AWS resource semantic representation within the controlled subset.
- Canonicalization/fingerprinting rules.
- Infrastructure dependency facts.
- Infrastructure-level security/functional facts available to DAIL.

25.2 What later documents own

- Identity decision algorithm → Document 07.
- Impact/invalidation algorithm → Document 08.
- Verification decision/evidence policy → Document 09.
- Persistence schema → Document 05.
- Testing strategy → Document 12.

26. Technical Non-Goals

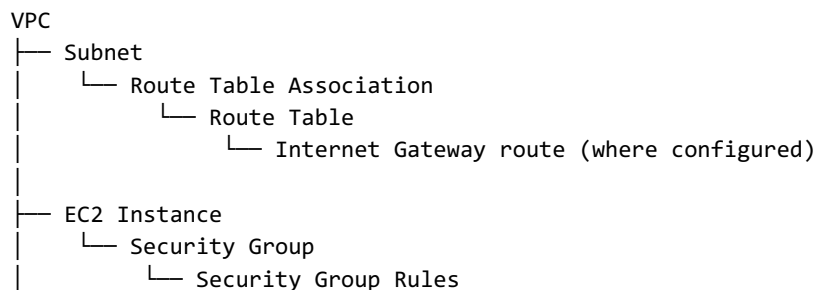
- Full Terraform interpreter.
- Complete AWS API semantic model.
- Support for every AWS networking construct.
- Provider-independent infrastructure semantics.
- Automatic inference of unknown cloud behavior.
- Production deployment of generated infrastructure.
- Treating Terraform configuration as proof of runtime behavior without an explicit verification contract.

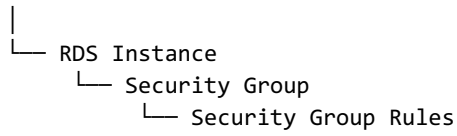
27. Final Terraform/AWS Directive

DAIL shall reason only over infrastructure semantics that are explicitly represented and tested in the controlled MVP model. Terraform syntax is an input format; the normalized model is the DAIL semantic boundary. AWS runtime state is an external verification source, not an implicit substitute for defined semantics.

When the system cannot safely establish a required infrastructure fact, it must preserve that uncertainty and prevent the downstream promotion path from treating the missing knowledge as proof.

Appendix A — MVP Resource Relationship Map





Functional invariant:

EC2 —TCP/5432→ RDS

Appendix B — Recommended Initial Terraform Fixture Shape

```
resource "aws_vpc" "main" { ... }
resource "aws_subnet" "app" { ... }
resource "aws_subnet" "db" { ... }
resource "aws_internet_gateway" "igw" { ... }
resource "aws_route_table" "public" { ... }
resource "aws_route_table_association" "app" { ... }
resource "aws_security_group" "app" { ... }
resource "aws_security_group" "db" { ... }
resource "aws_security_group_rule" "app_ssh_public" { ... }
resource "aws_security_group_rule" "app_to_db_5432" { ... }
resource "aws_instance" "app" { ... }
resource "aws_db_instance" "database" { ... }
```

Exact fixture values, CIDRs, credentials, instance sizing, and database settings must be defined in version-controlled fixtures and must not be embedded as undocumented assumptions in the core engine.

Appendix C — Source Baseline

This specification refines the requirements and functional contracts already established in Documents 01–03 and the project technical engineering baseline. It intentionally preserves the controlled AWS/Terraform subset, deterministic normalization, explicit dependency representation, canonical security rules, EC2–RDS TCP/5432 functional scenario, and conservative unsupported/uncertain semantics.